

Ledger

Software Requirements Specification

SMS-Native Personal Finance Agent for Informal & Multi-Bank Markets

Version 1.0 | Prepared for: Portfolio / Hiring Review | Status: Draft for Build

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for Ledger, a backend-centric personal finance agent that ingests bank and mobile-money SMS notifications directly from a user's Android device, parses and categorizes the underlying transactions without manual data entry, and produces a weekly, narrative spending report. This document is written to IEEE 830-style conventions, adapted for a solo-engineer, agile build.

1.2 Scope

Ledger consists of three cooperating components: (1) a thin Android client that captures qualifying SMS messages and forwards them to the backend, (2) a Go backend that ingests, parses, categorizes, and reasons over transaction data via an LLM-backed agent orchestrator, and (3) a web dashboard that presents balances, categorized spending, and the weekly report. The system explicitly targets banking/mobile-money SMS formats used in Ethiopia (e.g. CBE, Telebirr, Awash, Dashen) as the primary ingestion path, with statement upload (PDF/CSV) as a universal fallback for platforms or users where SMS capture is unavailable.

1.3 Definitions, Acronyms, and Abbreviations

- **SMS Capture** — the Android client's interception of an incoming SMS via a registered broadcast receiver.
- **Ingestion** — the backend process of receiving a raw message or document and turning it into a candidate transaction record.
- **Categorization** — assignment of a transaction to a spending category (e.g. Transport, Airtime, Rent, Transfer-Out).
- **Agent Orchestrator** — the backend component that decides when to ask the user a clarifying question, flag an anomaly, or generate a report, and calls the LLM to do so.
- **Sender ID** — the alphanumeric or numeric identifier a carrier attaches to an SMS, used to identify the originating bank/telecom.
- **Sideload** — installing an Android APK by direct file transfer rather than through the Google Play Store.

1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications (structural reference only).

- Android Developer Documentation — BroadcastReceiver, SmsManager, Runtime Permissions.
- Target bank/telecom SMS format samples (CBE, Telebirr, Awash, Dashen) — collected during implementation.

1.5 Overview

Section 2 describes the product at a high level. Section 3 enumerates functional requirements by subsystem. Section 4 defines external interfaces. Section 5 defines non-functional requirements. Section 6 covers distribution, compliance, and other constraints. An appendix provides a requirements traceability summary.

2. Overall Description

2.1 Product Perspective

Ledger is a new, standalone system with no dependency on existing budgeting products. It deliberately avoids manual expense logging — its central design premise is that the transaction record already exists as an SMS notification the user receives and normally ignores; Ledger's job is to read what already happened rather than ask the user to record it.

2.2 Product Functions (Summary)

- Capture qualifying bank/telecom SMS messages on the user's device and transmit them securely to the backend.
- Parse heterogeneous SMS formats into a normalized transaction schema (amount, direction, counterparty, timestamp, channel).
- Categorize transactions automatically; learn from user corrections over time.
- Detect ambiguous or anomalous transactions and raise a clarifying question to the user.
- Generate a weekly report combining a visual summary with a short, plain-language narrative explanation.
- Present balances, category breakdowns, and report history on a web dashboard.
- Accept PDF/CSV statement uploads as an alternative ingestion path.

2.3 User Classes and Characteristics

ID	Requirement	Priority
UC-1	Primary user — an Android-owning individual (student, informal worker, salaried employee) who wants passive spending visibility without manual logging.	High
UC-2	Fallback user — any user (including iOS) without SMS capture, who uploads a statement instead.	Medium
UC-3	Administrator/operator (the engineer) — manages parser definitions per bank format and monitors system health.	Medium

2.4 Operating Environment

Android client: Android 8.0+ devices, distributed via direct APK download (sideloading) rather than the Play Store, given target-market distribution constraints. Backend: containerized Go services deployable to any Linux host or managed platform, with PostgreSQL as the system of record. Dashboard: any modern mobile or desktop browser.

2.5 Design and Implementation Constraints

- SMS inbox access is an Android-only capability; iOS provides no equivalent API. This is a platform limitation, not a design choice, and is addressed via the statement-upload fallback (see FR-7).
- Bank/telecom SMS formats are undocumented and subject to silent change; the parser layer must be added-to, not rewritten, as new formats are encountered (see FR-3).
- Sideloaded distribution means no centralized auto-update mechanism; the client must support a manual or in-app update check.
- Financial data sensitivity requires encryption in transit and at rest, and a data-minimization approach to what leaves the device (see Section 5.2).

2.6 Assumptions and Dependencies

- The user grants SMS-read permission on first launch and understands why it is requested.
- The device has intermittent, not necessarily constant, internet connectivity; the client queues unsent messages.
- An external LLM API is reachable from the backend for narrative generation and clarifying-question phrasing.

3. System Features (Functional Requirements)

3.1 FR-1 — SMS Capture & Local Filtering (Android Client)

The client registers a broadcast receiver for incoming SMS. On receipt, it checks the sender ID against a locally stored allow-list of known bank/telecom identifiers. Non-matching messages are discarded on-device and never transmitted, minimizing data exposure and reinforcing user trust.

ID	Requirement	Priority
FR-1.1	Request SMS_READ / RECEIVE_SMS permission with an explicit, plain-language rationale shown before the OS prompt.	Must
FR-1.2	Maintain a local, updatable allow-list of bank/telecom sender IDs; default list ships with the app.	Must
FR-1.3	Forward only allow-listed messages to the backend over HTTPS; queue locally if offline and retry on reconnect.	Must
FR-1.4	Never persist or transmit non-matching (personal) SMS content.	Must
FR-1.5	Provide an in-app log of what was captured and sent, for user transparency.	Should

3.2 FR-2 — Ingestion API (Backend)

ID	Requirement	Priority
FR-2.1	Expose an authenticated endpoint to receive a raw SMS payload (sender ID, body, received timestamp, device ID).	Must
FR-2.2	Expose an authenticated endpoint to receive an uploaded statement file (PDF or CSV).	Must
FR-2.3	De-duplicate messages/transactions idempotently (a message re-sent after a retry must not create a duplicate transaction).	Must
FR-2.4	Persist the raw payload alongside the parsed result, to allow reprocessing when a parser is corrected.	Should

3.3 FR-3 — Parsing Engine

Each supported bank/telecom format is implemented as an independent parser module conforming to a shared interface, so a new bank format can be added without modifying existing parsers.

ID	Requirement	Priority
FR-3.1	Identify the source format from sender ID and/or message shape before attempting extraction.	Must
FR-3.2	Extract amount, direction (debit/credit), counterparty/reference, timestamp, and running balance where present.	Must
FR-3.3	Route messages that match no known parser to a review queue rather than failing silently.	Must

ID	Requirement	Priority
FR-3.4	Support statement (PDF/CSV) parsing via a comparable per-bank plugin interface.	Must

3.4 FR-4 — Categorization & Learning Loop

ID	Requirement	Priority
FR-4.1	Apply rule-based categorization using counterparty name, keyword, and historical mapping.	Must
FR-4.2	Persist user corrections to categorization and apply them to future matching transactions automatically.	Must
FR-4.3	Track a confidence level per categorized transaction to drive clarifying-question logic (FR-5).	Should

3.5 FR-5 — Agent Orchestrator

This is the reasoning layer distinguishing Ledger from a plain categorizer: it decides when to involve the user and when to act autonomously, and it improves its own future behavior from the answers it receives.

ID	Requirement	Priority
FR-5.1	When categorization confidence is below threshold, generate a short clarifying question and queue it for the user.	Must
FR-5.2	Apply the user's answer to the pending transaction and store it as a reusable rule (FR-4.2).	Must
FR-5.3	Detect week-over-week anomalies per category (e.g. a spend spike) and flag them for inclusion in the weekly report.	Must
FR-5.4	Generate the weekly narrative summary via LLM call, grounded strictly in that user's own transaction data for the period.	Must

3.6 FR-6 — Weekly Report Generation

ID	Requirement	Priority
FR-6.1	Run on a scheduled worker once per week per user (configurable day/time).	Must
FR-6.2	Include a category breakdown chart, week-over-week comparison, and a short written explanation of notable changes.	Must
FR-6.3	Deliver the report via the dashboard and, optionally, email.	Should

3.7 FR-7 — Statement Upload Fallback

ID	Requirement	Priority
FR-7.1	Accept PDF/CSV statement upload from the web dashboard for any user, independent of platform.	Must
FR-7.2	Merge uploaded-statement transactions with any SMS-sourced transactions for the same account without duplication.	Should

3.8 FR-8 — Dashboard & Authentication

ID	Requirement	Priority
FR-8.1	Provide registration/login (email + password, or OTP) independent of the Android client.	Must
FR-8.2	Display current balance estimate, category breakdown, transaction list, and report history.	Must
FR-8.3	Surface pending clarifying questions from FR-5.1 for the user to answer.	Must
FR-8.4	Enforce per-user data isolation (multi-tenancy) at the data-access layer.	Must

4. External Interface Requirements

4.1 User Interfaces

Android client: a minimal single-screen app (permission grant, capture status, allow-list view). Web dashboard: responsive layout covering balance summary, transaction list, category charts, and report history.

4.2 Hardware Interfaces

None beyond the Android device's standard SMS radio/inbox, accessed through the OS-provided API.

4.3 Software Interfaces

- LLM provider API (chat/completion endpoint) for narrative generation and clarifying-question phrasing.
- PostgreSQL for transactional storage.
- Object storage (local disk or S3-compatible) for retained raw statement uploads.

4.4 Communication Interfaces

All client-backend and dashboard-backend communication occurs over HTTPS/TLS 1.2+. See the separate API Contract document for endpoint-level detail.

5. Non-Functional Requirements

5.1 Performance

- A single SMS ingestion request should be parsed and persisted in under 500ms at the 95th percentile, excluding LLM calls.
- Weekly report generation for a single user should complete in under 10 seconds, dominated by the LLM call.

5.2 Security & Privacy

- Only allow-listed financial SMS content is ever transmitted off-device (see FR-1.4).
- All data encrypted in transit (TLS) and at rest (database-level encryption).
- Raw message/statement content is scoped strictly to the owning user; no cross-user access path exists.
- The app must clearly disclose, in plain language, what it reads and why, before requesting SMS permission.

5.3 Reliability & Availability

The ingestion path must tolerate intermittent connectivity: the Android client queues and retries; the backend treats ingestion as idempotent (FR-2.3) so retried sends never duplicate transactions.

5.4 Scalability

The parser and categorization layers are stateless and horizontally scalable; the scheduled report worker is designed to process users in parallel batches rather than serially.

5.5 Maintainability

New bank/telecom SMS formats must be addable as an isolated parser module without modifying existing parsers or the ingestion API contract (open/closed principle applied at the parser-interface boundary).

5.6 Localization

Parsing and categorization must tolerate mixed-language content (English/Amharic merchant and counterparty names) without requiring a full translation layer for core functionality.

6. Other Requirements

6.1 Distribution Constraints

The Android client is distributed via direct APK download rather than the Google Play Store. The system must therefore include an in-app or landing-page mechanism for checking and retrieving updates, and must clearly instruct users through the OS "install from unknown sources" step.

6.2 Data Retention

Raw SMS/statement payloads are retained to allow re-parsing after a parser correction, subject to a defined retention window; the user may request deletion of their account and associated data.

Appendix A — Requirements Traceability Summary

ID	Requirement	Priority
FR-1	SMS Capture & Local Filtering	Android client
FR-2	Ingestion API	Go backend
FR-3	Parsing Engine	Go backend
FR-4	Categorization & Learning Loop	Go backend
FR-5	Agent Orchestrator	Go backend + LLM
FR-6	Weekly Report Generation	Go backend (scheduled worker)
FR-7	Statement Upload Fallback	Go backend + Dashboard
FR-8	Dashboard & Authentication	Web frontend + Go backend